SOFTWARE REQUIREMENTS SPECIFICATION

for

Smart Airport Management System (SAMS)

Version 1.0

Role- and Department-Based Expanded Scope

Prepared by Group 5:

Ridhi Mahajan (1024240048)
Priyanka (1024240053)
Diya Saini (1024240057)
Ishika Jain (1024240066)

Submitted to: Ms. Anushka

Computer Science and Engineering Department
Thapar Institute of Engineering and Technology, Patiala

October 14, 2026

Contents

1 Introduction **5**
- 1.1 Purpose . . . 5
- 1.2 Intended Audience and Reading Suggestions . . . 5
- 1.3 Project Scope . . . 5
- 1.4 Definitions, Acronyms and Abbreviations . . . 6

2 Overall Description **8**
- 2.1 Product Perspective . . . 8
- 2.2 User Classes and Characteristics . . . 8
- 2.3 Product Functions . . . 11
- 2.4 Operating Environment . . . 13
- 2.5 Design and Implementation Constraints . . . 13
- 2.6 Example Workflow: A Baggage Issue . . . 13
- 2.7 User Documentation . . . 14
- 2.8 Assumptions and Dependencies . . . 14

3 System Features **16**
- 3.1 Description and Priority . . . 16
- 3.2 Functional Requirements . . . 17
- 3.3 Key Features Summary . . . 23

4 Other Nonfunctional Requirements **25**
- 4.1 Performance Requirements . . . 25
- 4.2 Safety and Reliability Requirements . . . 25
- 4.3 Security and Privacy Requirements . . . 25
- 4.4 Software Quality Attributes . . . 26
- 4.5 Business Rules . . . 26

5 Other Requirements **28**
- 5.1 Database Requirements . . . 28
- 5.2 External Interface Requirements . . . 30
- 5.3 Verification and Traceability . . . 31
- 5.4 Delivery Phases . . . 32
- 5.5 Acceptance Criteria for the Initial Expanded MVP . . . 32
- 5.6 Future Enhancements . . . 32

List of Figures

2.1 System architecture 8
2.2 Context diagram (DFD Level 0) 9
2.3 Use-case diagram 11
2.4 Data flow diagram (Level 1) 12
2.5 Sequence diagram — reporting a baggage issue and acknowledging the notification 14

3.1 State diagrams — action item escalation (left) and change request approval (right) 24

5.1 Class diagram of the main data entities 29

List of Tables

2.3 Proposed department responsibilities 10

3.1 System features and priorities 16
3.4 Proposed permission matrix (C = create, R = read, U = update, D = delete, A = approve; R_s = relevant summaries only; own = own records only; – = no access) 18
3.8 Example impact rules (configurable) 21

5.1 Proposed new collections 28

1 Introduction

1.1 Purpose

This document specifies the software requirements for the **Smart Airport Management System (SAMS)**, Version 1.0. SAMS is a web application that centralizes selected airport operational records and coordinates follow-up work between authorized airport departments.

SAMS builds on an existing working prototype, which gives a single administrator a central interface for flights, luggage, visitors, staff and transportation. This specification extends the prototype into a *role- and department-based* system: each department works in its own workspace on the records it owns, other departments see the information they need for coordination, and significant operational events automatically notify the right people, create follow-up tasks and leave an accountable history.

This SRS is a requirements specification. It does not assert that the newly described features are already implemented; the status of every requirement is stated explicitly.

1.2 Intended Audience and Reading Suggestions

- **Project reviewers and stakeholders** — to assess scope, feasibility and verifiability. Chapters 1 and 2 give the complete picture.
- **Development team (Group 5)** — to plan and implement the system. Chapter 3 (System Features) and Chapter 5 (data, interfaces and phases) are the most relevant.
- **Testers** — to derive test cases from requirement IDs (Section 5.3).
- **Future maintainers** — to understand roles, departments, data ownership and extension points.

Readers new to the project should start with the one-line summary below, then read Section 2.2 (users and departments) and Section 2.6 (example workflow).

1.3 Project Scope

Airport operations involve many departments — flight operations, baggage handling, ground transport, maintenance and passenger services — that depend on each other's information. When each department keeps isolated records, a problem in one area (for example, a baggage issue on a flight) is communicated late or inconsistently, nobody is clearly responsible for the follow-up, and there is no reliable record of who did what and when.

SAMS addresses this with one web application and one shared backend and database. Public flight and transport information is available without login. Passengers may optionally create an account, register their journeys and receive targeted notifications. Staff and administrators sign in through a common entry point and are redirected to the workspace appropriate to their role and department.

Objectives

1. Centralize flight, luggage/baggage, visitor, staff and transportation records, retaining the existing prototype modules where practical.

2. Enforce server-side authorization by **role, department, resource scope and permitted fields**.
3. Provide separate role-aware workspaces within **one** React application, not separate codebases or login systems.
4. Link operational events to **targeted departmental notifications and action items**.
5. Provide an **incident and audit timeline** showing important changes, decisions, acknowledgements and outcomes.
6. **Escalate overdue** high-priority tasks to the responsible supervisor under configurable rules.
7. Provide passenger-specific journey and baggage notifications **without exposing another passenger's private information**.

In Scope (initial expanded MVP)

- Existing authentication and management modules from the prototype, preserved and extended.
- Public, read-only flight, announcement and transport information.
- Passenger accounts, registered journeys and targeted in-app notifications, using demonstration data.
- Fully implemented workspaces for the first three departments: **Flight Operations, Baggage Handling and Ground Transport**.
- **Maintenance and Passenger Services** represented in the design and added as time permits (phased).
- Central administrator workspace, supervisor permissions, approval workflow, event processing, action-item tracking, audit timeline and overdue-task escalation.
- Testing with seeded demonstration records and one complete cross-department incident scenario.

Out of Scope (initial expanded MVP)

- Live airline, airport, RFID, air-traffic or baggage-scanner integrations.
- Real ticket booking, payments, check-in or boarding-pass issuance.
- Real airline operational decisions such as guaranteed connection rebooking.
- SMS delivery and production notification providers (email may be a later extension).
- AI-based prediction, microservices and large-scale production deployment.

1.4 Definitions, Acronyms and Abbreviations

Term	Meaning
SAMS / SRS	Smart Airport Management System / Software Requirements Specification
Role	What a user is allowed to do (e.g. Department Staff, Department Supervisor).

Term	Meaning
Department	The operational area a staff user belongs to (e.g. Baggage Handling). A department is *not* a role.
Workspace	The set of pages and controls a signed-in user sees, determined by role and department.
Owning department	The department responsible for authorized changes to a record type.
Event	A persisted record that a significant operational action occurred.
Impact rule	A configured rule that maps an event to the departments/users to notify and the tasks to create.
Notification	An in-app message to a user or department about an event.
Acknowledgement	Confirmation that a notification was seen. It does *not* complete a task.
Action item	A follow-up task assigned to a department or person, with status and due time.
Sensitive change	A change that must be approved before it alters the authoritative record (Section 3.2.6).
Change request	A pending sensitive change awaiting an approval decision.
Incident	A coordinated operational problem and all events, tasks and decisions linked to it.
Audit timeline	Chronological presentation of recorded actions and changes for an incident or record.
Escalation	Alerting a supervisor (and, by rule, central administration) when a task passes its deadline.
Public information	Information approved for viewing by unauthenticated users.
Visitor pass	A record in the retained Visitors module. Not to be confused with a *public user*.
RBAC / JWT	Role-Based Access Control / JSON Web Token
MVP	Minimum Viable Product — the initial expanded version described in this SRS.

2 Overall Description

2.1 Product Perspective

SAMS evolves the existing prototype rather than replacing it. It keeps the layered (three-layer) architecture and the technology stack, and adds new modules on top of the existing ones. The system remains a **modular monolith**: all modules share one backend and one database, while access is controlled by the API.

1. **Presentation layer** — one React single-page application containing the public portal, the passenger *My Journey* pages, the department workspaces and the administrator workspace.
2. **Application/API layer** — a Node.js/Express REST API. Every protected request passes through authentication and authorization middleware before reaching controllers and services. New services handle events, impact rules, notifications, action items, approvals, escalation and audit logging.
3. **Data layer** — MongoDB accessed through Mongoose. The browser never accesses the database directly.

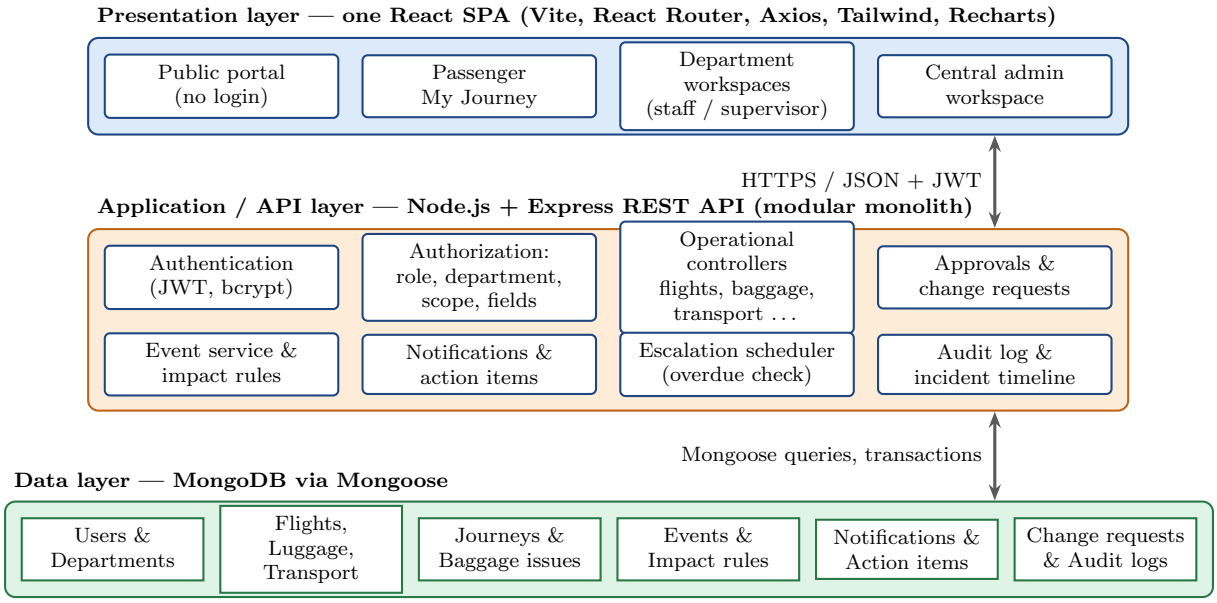


Figure 2.1: System architecture

Figure 2.2 shows SAMS in context with its users.

2.2 User Classes and Characteristics

Stakeholders

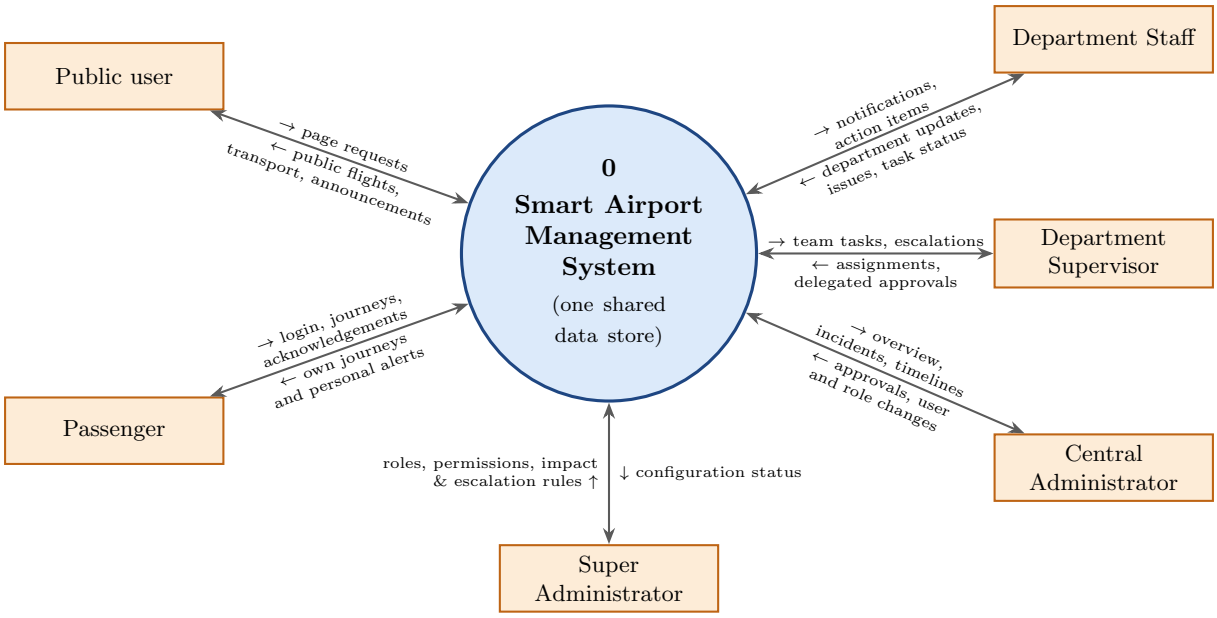


Figure 2.2: Context diagram (DFD Level 0)

Stakeholder	Interest in the system
Airport management	One reliable operational picture; accountability for incidents and follow-up.
Airport departments	Timely, relevant information from other departments; clear ownership of their own records.
Department supervisors	Visibility of team tasks; early warning when tasks become overdue.
Central administration	Cross-department overview, approvals, and control of user accounts and roles.
Passengers	Accurate public information and personal alerts about their own journeys and baggage.
Development team (Group 5)	A clear, testable specification to build and demonstrate.
Project reviewers	Assessing requirements, design, implementation and testing.

Roles and Access

SAMS applies role-based access control (RBAC), extended with departments. A **role** defines what a person may do; a **department** defines the operational area they belong to. A user account has one role and, for staff and supervisors, one department. For example, a baggage employee is *Department Staff* in *Baggage Handling*; a baggage supervisor belongs to the same department with additional supervisory permissions.

User class	Access method	Responsibilities and access
Public user	No login	Views published flights, announcements and transport information only.

User class	Access method	Responsibilities and access
Passenger	Optional passenger sign-in	Public information plus own registered journeys, own baggage details and own notifications. No access to internal records or other passengers' data.
Department Staff	Common staff login	Creates and updates authorized records and fields of the assigned department; views relevant shared summaries; receives and acknowledges notifications; works on assigned tasks.
Department Supervisor	Same login, added permissions	Staff permissions plus team task assignment, departmental oversight, receipt of overdue-task escalations, and approval of explicitly delegated changes.
Central Administrator	Same login, role-based redirect	Cross-department overview, incident monitoring, designated approvals, account and role/department administration within granted permissions, rule-based escalations.
Super Administrator	Restricted technical access	Global roles, permissions and system configuration (impact and escalation rules). Not an operational department; may be omitted from the initial demonstration.

The role values used in the prototype are mapped to the new user classes as follows, so that existing accounts keep working: `admin` → Central Administrator; `airline_staff` → Department Staff (Flight Operations); `staff` → Department Staff (Baggage Handling); `transport_operator` → Department Staff (Ground Transport); `passenger` → Passenger. The prototype's `visitor` login role is retired, because public information no longer requires an account.

Departments and Responsibilities

Seeing a record does not automatically allow editing it. The department that owns a record type is responsible for authorized changes to it; other departments see only the information they need for coordination. The use cases of all user classes are shown in Figure 2.3, drawn in UML use-case notation.

Table 2.3: Proposed department responsibilities

Department	Can create or update	Can view
Flight Operations	Authorized flight status, schedule, gate, and operational decisions	Relevant baggage, transport, maintenance, and incident information
Baggage Handling	Baggage records, baggage incidents, handling status, and baggage follow-up tasks	Relevant flight information and issues affecting baggage operations
Ground Transport	Transport services, routes, availability, and transport disruptions	Relevant flights and incidents affecting transport

Department	Can create or update	Can view
Maintenance (phased)	Maintenance issues, repair status, equipment availability, and repair tasks	Relevant affected departments and service information
Passenger Services (phased)	Passenger-facing announcements, service enquiries, and authorized passenger updates	Published flight information and approved summaries of relevant incidents
Central Administration	Authorized account management, designated approvals, and operational oversight	Cross-department overview, incidents, approvals, and action status

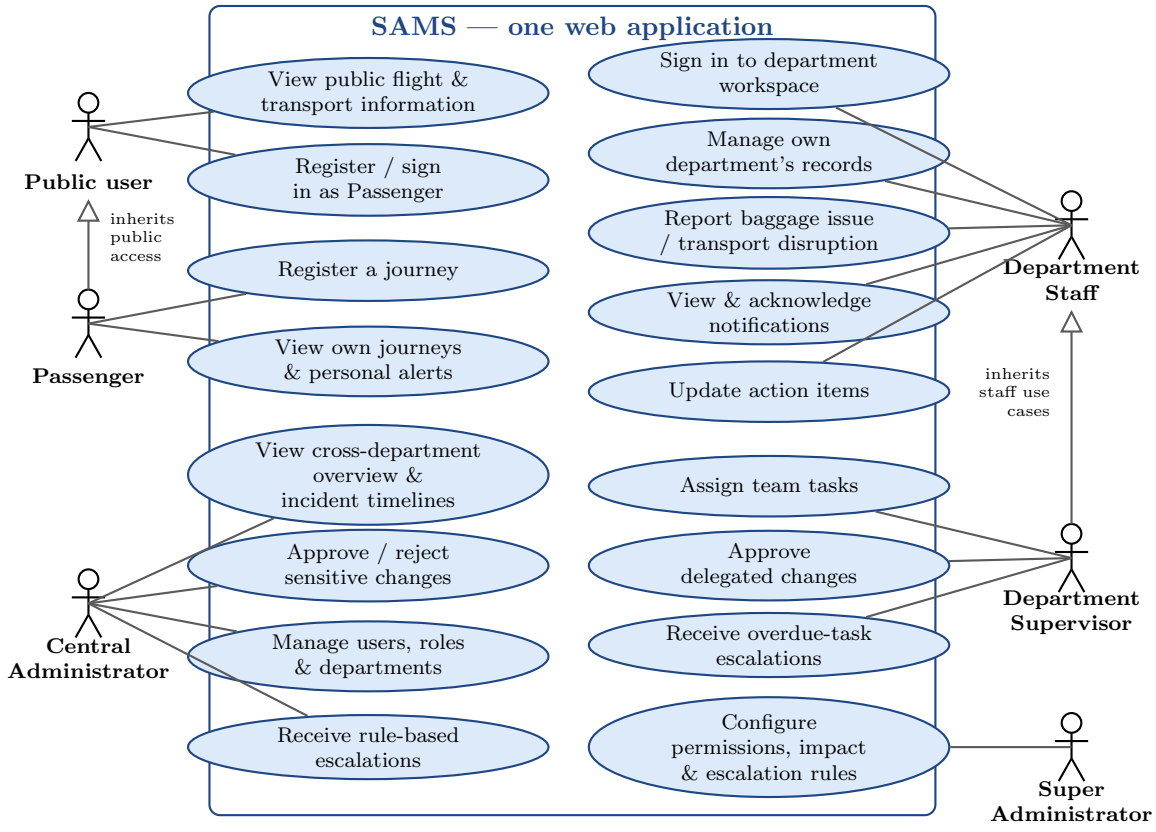


Figure 2.3: Use-case diagram

2.3 Product Functions

At a high level, SAMS provides the following functions (detailed in Chapter 3):

- **Authentication and account management** — passenger self-registration, common staff login, role-based redirection, administrator-controlled roles and departments.
- **Authorization and record scope** — checks by role, department, resource and field on every protected request.
- **Operational management** — flights, luggage/baggage, baggage issues, visitors, staff and transportation.

- **Public portal and passenger journeys** — public information without login; optional personal journeys and alerts.
- **Events, notifications and action items** — automatic, rule-based coordination between departments.
- **Approval workflow** — sensitive changes stay pending until approved.
- **Incident and audit timeline** — the full, persisted history of each incident.
- **Overdue-task escalation** — supervisor alerts and rule-based escalation to central administration.
- **Role-aware dashboards and reports** — database-derived values; demonstration data clearly labelled.

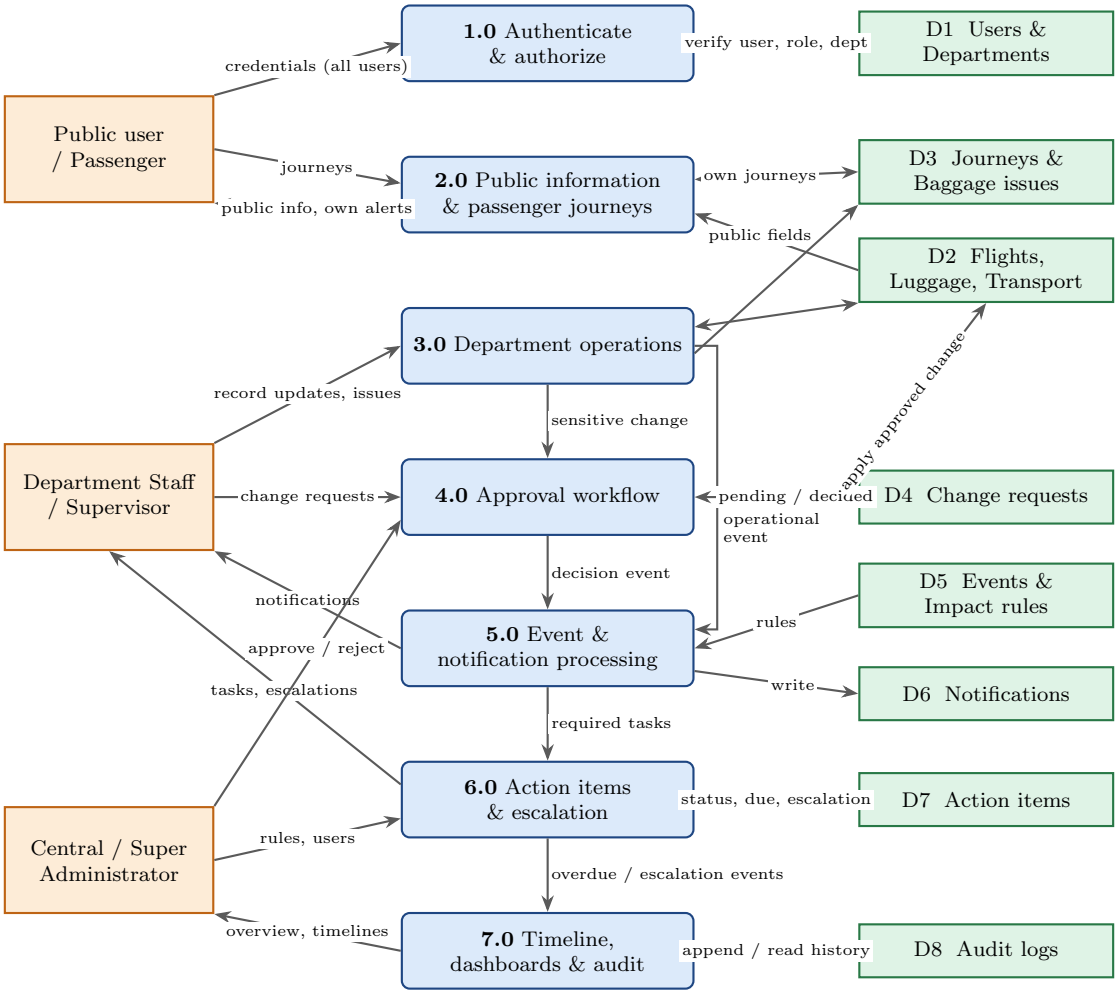


Figure 2.4: Data flow diagram (Level 1)

2.4 Operating Environment

Component	Environment
Client	Any modern desktop or mobile browser (Chrome, Firefox, Edge, Safari) with JavaScript enabled; responsive layout.
Server runtime	Node.js (LTS) running Express.js.
Database	MongoDB (local instance or MongoDB Atlas) accessed via Mongoose. A replica-set deployment (e.g. Atlas) is preferred so that multi-document transactions are available (FR-5.7).
Operating system	Platform-independent: Windows, Linux or macOS.
Network	HTTP(S) between browser and API; configurable API base URL.

2.5 Design and Implementation Constraints

- **Technology stack:** the prototype's technology stack is retained (React, Vite, React Router, Axios, Tailwind CSS, Recharts; Node.js, Express, Mongoose, JWT, bcryptjs; MongoDB; Git/GitHub) unless a blocker is found.
- **Architecture:** a modular monolith with one frontend, one API and one database — not microservices and not separate applications per department.
- **Preserve working code:** existing working login and CRUD features shall be preserved and extended, not needlessly replaced.
- **Data:** no live airport or airline feed is assumed; demonstrations use seeded or manually entered data.
- **Secrets:** database credentials and JWT secrets are held in environment variables and are never committed to the repository.
- **Notifications:** in-app only for the MVP; no SMS or external delivery providers.
- **Team and schedule:** a four-member development team working to a fixed delivery schedule; Maintenance and Passenger Services are therefore phased.
- **Cost:** only free and open-source tools and free-tier hosting.

2.6 Example Workflow: A Baggage Issue

The following scenario is the reference cross-department workflow for the MVP and the basis of the end-to-end acceptance test. Figure 2.5 shows it as a UML sequence diagram.

1. Baggage Handling staff report a baggage issue and link it to a flight.
2. SAMS stores the issue and records an operational event, in the same transaction where supported.
3. Impact rules identify the relevant recipients, such as Baggage Handling and Flight Operations.
4. Relevant people receive in-app notifications, and follow-up action items are created.
5. Flight Operations assesses any flight impact. A baggage report does *not* automatically change the flight schedule.

6. If a sensitive change is needed (e.g. a delay), it follows the approval policy and the decision is recorded.
7. Only relevant passengers receive personal alerts; public pages show approved public information only.
8. Staff record follow-up and resolution; the incident timeline preserves the full history.

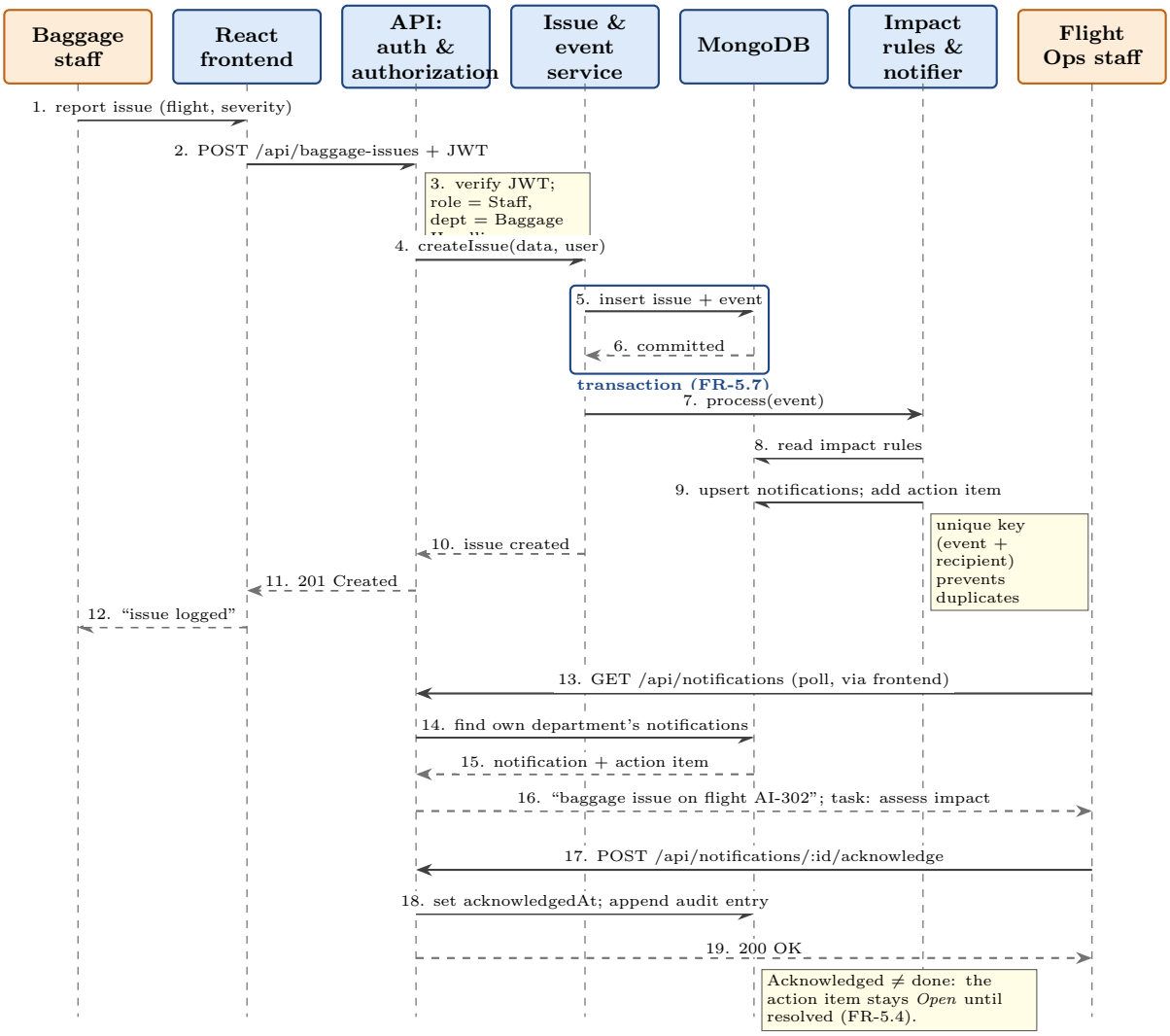


Figure 2.5: Sequence diagram — reporting a baggage issue and acknowledging the notification

2.7 User Documentation

The repository will contain a `README` covering installation, environment configuration, seeding demonstration data and running the backend and frontend, plus a short user guide for each workspace. This SRS and the project report serve as the requirements and design documentation.

2.8 Assumptions and Dependencies

- Passenger-specific notifications depend on registered journeys and a reliable link between passenger accounts, flights and baggage records.

- A flight delay alone does not prove that a connecting passenger will miss a connection; the system must not present an unconfirmed outcome as fact.
- Public endpoints expose only approved public fields; internal incidents and personal data remain protected.
- The Central Administrator account and the department list are created (seeded) before the system is demonstrated.
- A MongoDB instance is available and reachable from the backend.
- New capabilities are planned requirements and must not be described as implemented until they are verified and tested.

3 System Features

3.1 Description and Priority

Priorities are **High** (essential for the MVP), **Medium** (important) and **Low** (desirable). Each functional requirement is written as a uniquely identified, verifiable "shall" statement, as recommended by ISO/IEC/IEEE 29148. Each requirement also carries an implementation status, checked against the current SAMS source code:

- **Implemented** — present and working in the current code base.
- **Partial** — partly present; the requirement states what is missing.
- **Planned** — new or extended requirement; not claimed as implemented.
- **Phased** — planned for after the first three departments work end to end.
- **To verify** — behaviour whose target value must be measured.

Table 3.1: System features and priorities

No.	Feature	Description	Priority
F1	Authentication and Accounts	Passenger self-registration, common staff login, role-based redirection, managed roles and departments.	High
F2	Authorization and Record Scope	Role, department, resource and field checks on the server.	High
F3	Operational Management	Flights, luggage, baggage issues, visitors, staff, transportation.	High
F4	Public Portal and Passenger Journeys	Public information without login; own journeys and alerts.	High
F5	Events, Notifications, Action Items	Rule-based targeted coordination between departments.	High
F6	Approval Workflow	Pending sensitive changes, approve/reject, urgent path.	High
F7	Incident and Audit Timeline	Chronological, persisted, protected history.	High
F8	Overdue-Task Escalation	Deadlines, supervisor alerts, rule-based escalation.	High
F9	Dashboards, Reports, Settings	Role- and department-aware views.	Medium

3.2 Functional Requirements

3.2.1 F1 — Authentication and Account Management

Stimulus/response: A user signs in on the common login page; the server verifies the credentials, issues a JSON Web Token (JWT), and the frontend opens the workspace for the verified role and department.

ID	Requirement	Priority	Status
FR-1.1	The system shall support login, logout and retrieval of the authenticated user's profile (`GET /api/auth/me`).	High	Implemented
FR-1.2	Public self-registration shall create Passenger accounts only; the server shall ignore or reject any client-supplied role.	High	Planned
FR-1.3	Privileged roles and department assignments shall be created and changed only by authorized administrators.	High	Planned
FR-1.4	The system shall keep the existing JWT and bcrypt-based authentication, with secrets held in environment configuration.	High	Implemented
FR-1.5	After a successful staff or administrator login, the frontend shall redirect the user to the workspace for the verified role and department.	High	Planned
FR-1.6	The backend shall reject invalid or expired tokens (implemented) and accounts marked inactive (planned: requires an `active` flag on users).	High	Partial
FR-1.7	The JWT shall carry the user's ID, role and department; the server shall reload the user's current role, department and active state for privileged operations so that revoked access takes effect immediately.	Medium	Planned

3.2.2 F2 — Authorization and Record Scope

Stimulus/response: Every protected request is checked for a valid token, then for role, department, record scope and permitted fields, before any data is read or changed. Role checks follow the RBAC model; department, record and field checks refine it.

ID	Requirement	Priority	Status
FR-2.1	Every protected API operation shall enforce authorization on the backend; hiding a frontend control alone shall not be relied on for security.	High	Implemented
FR-2.2	Authorization shall consider role, department, resource scope and permitted fields. Role checks are currently implemented; department, scope and field checks are planned.	High	Partial

ID	Requirement	Priority	Status
FR-2.3	A department shall be able to edit only the record types and fields it owns (Table 3.4); read access shall not imply edit access.	High	Planned
FR-2.4	Passengers shall access only their own journeys, their own baggage information and explicitly public data.	High	Planned
FR-2.5	Users shall not be able to change their own role or department by submitting modified request fields.	High	Planned
FR-2.6	Unauthorized requests shall return HTTP 401 (not authenticated) or 403 (not permitted) without changing stored data.	High	Implemented
FR-2.7	The permission matrix in Table 3.4 shall be defined in one server-side configuration so that it can be reviewed and tested as a whole.	Medium	Planned

Table 3.4: Proposed permission matrix (C = create, R = read, U = update, D = delete, A = approve; R_s = relevant summaries only; own = own records only; – = no access)

Resource	Public	Passenger	Flight Ops	Baggage	Gnd Transport	Maint.*	Pass. Serv.*	Central Admin
Flights	R[3]	R[3]	CRUD[1]	R	R	R	R	R, A
Luggage / baggage	–	own	R_s	CRUD	–	–	–	R
Baggage issues	–	–	R_s	CRU	–	–	R_s	R
Transportation	R[3]	R[3]	R_s	–	CRUD[1]	–	R	R, A
Maintenance issues*	–	–	R_s	–	–	CRU	–	R
Announcements*	R[2]	R[2]	R	R	R	R	CRU	R, A
Passenger journeys	–	own	–	–	–	–	R_s	R
Visitors (passes)	–	–	–	–	–	–	–	CRUD
Staff records	–	–	–	–	–	–	–	CRUD
Users, roles, departments	–	own profile	–	–	–	–	–	CRU
Notifications	–	own	own dept	own dept	own dept	own dept	own dept	R
Action items	–	–	dept	dept	dept	dept	dept	R
Audit logs	–	–	–	–	–	–	–	R

* Phased departments. [1] Sensitive changes (cancellation, departure-time change over 60 minutes, deletion) become change requests approved by Central Administration (Section 3.2.6). [3] Allowlisted public fields only. [2] Published announcements only. Supervisors have their department's permissions plus task assignment and delegated approval. Super Administrator manages roles, permissions and rules and is not shown as an operational column.

3.2.3 F3 — Operational Management

ID	Requirement	Priority	Status
FR-3.1	The system shall retain flight CRUD, search and status values from the prototype. Creating, updating and deleting flights shall be restricted to Flight Operations; sensitive flight changes require Central Administration approval (F6).	High	Partial
FR-3.2	The system shall retain luggage management and link each luggage record to an existing flight (existence check planned) and, where supported, to the owning passenger account (planned).	High	Partial

ID	Requirement	Priority	Status
FR-3.3	The system shall retain visitor-pass management and its validation rules (unique pass ID, validity period, status values). Ownership moves to Central Administration (Staff currently also has full access).	Medium	Partial
FR-3.4	The system shall retain staff records, and restrict account, role and department administration to authorized administrators.	High	Partial
FR-3.5	The system shall retain transportation management (Cab/Shuttle, route, scheduled time, status) and validate that the referenced flight exists.	High	Partial
FR-3.6	Baggage Handling shall be able to record a **baggage issue** linked to a flight, with description, severity (Low/Medium/High/Critical), status (Open/Investigating/Resolved), reporter, timestamps and optional references to affected luggage or passengers.	High	Planned
FR-3.7	Ground Transport shall be able to record a transport disruption against a scheduled service and update service availability.	Medium	Planned
FR-3.8	Maintenance issues and Passenger Services announcements shall be added after the first three departments work end to end.	Medium	Phased

3.2.4 F4 — Public Portal and Passenger Journeys

ID	Requirement	Priority	Status
FR-4.1	Unauthenticated users shall be able to view approved public flight, announcement and transport information through dedicated public endpoints.	High	Planned
FR-4.2	A Passenger shall be able to register one or more journeys made of one or more flight segments, using demonstration flight and connection data.	High	Planned
FR-4.3	The system shall display journey-specific flight and baggage notifications only to the passenger who owns the journey.	High	Planned
FR-4.4	The system shall not assume that a flight delay causes a missed connection; where impact is uncertain, it shall show a *potential impact* state until the outcome is confirmed.	High	Planned
FR-4.5	Public endpoints shall return only allowlisted public fields and shall never expose another passenger's baggage details or internal incident information.	High	Planned

FR-4.6	The MVP shall use in-app notifications and demonstration data only; external airline feeds and SMS are out of scope.	Medium	Planned

3.2.5 F5 — Events, Notifications and Action Items

Stimulus/response: A qualifying update (e.g. a baggage issue, a gate change) creates an event; impact rules decide who is notified and which follow-up tasks are created.

ID	Requirement	Priority	Status
FR-5.1	A qualifying operational update shall create a persisted event with event type, entity reference, actor, department, timestamp and correlation ID.	High	Planned
FR-5.2	Configured impact rules shall identify the departments and users to notify and whether an action item is required (examples in Table 3.8).	High	Planned
FR-5.3	The system shall create in-app notification records for intended recipients and shall not create duplicates when an event is processed again.	High	Planned
FR-5.4	Notifications and action items shall be separate records; acknowledging a notification shall not complete an action item or resolve an incident.	High	Planned
FR-5.5	Users shall be able to view and acknowledge their own and their department's notifications.	High	Planned
FR-5.6	The system shall create action items with department, optional assignee, priority, status (Open/In Progress/Resolved), due time and resolution note.	High	Planned
FR-5.7	An operational change and its event shall be committed together using a database transaction where supported; otherwise an explicit, tested recovery strategy shall prevent silent event loss.	High	Planned
FR-5.8	Supervisors shall be able to assign and reassign their department's action items.	Medium	Planned

Table 3.8: Example impact rules (configurable)

Event	Notified	Action item created
Baggage issue reported (High/Critical)	Baggage Handling supervisor; Flight Operations	Flight Operations: assess flight impact (due 30 min)
Flight delayed	Ground Transport; Baggage Handling; passengers with a journey on the flight	Ground Transport: adjust pick-ups (due 30 min)
Gate changed	Baggage Handling; Passenger Services*; passengers on the flight	None
Flight cancellation requested	Central Administration (approval)	Approval decision (due 15 min)
Transport disruption	Flight Operations; Passenger Services*	None
Maintenance issue* affecting a gate	Flight Operations; Ground Transport	Flight Operations: reassign gate if needed

* Phased departments.

3.2.6 F6 — Approval Workflow

Sensitive changes in the MVP are: cancelling a flight; changing a departure time by more than 60 minutes; deleting a flight or transport record; and assigning or changing a privileged role or department. Other valid, authorized changes are **routine** and apply immediately.

ID	Requirement	Priority	Status
FR-6.1	Routine changes shall be applied immediately when the user is authorized and the change is valid.	High	Planned
FR-6.2	Sensitive changes shall be stored as pending change requests and shall not alter the authoritative record until approved.	High	Planned
FR-6.3	Approval and rejection shall record the reviewer, decision, timestamp and, for rejections, a reason.	High	Planned
FR-6.4	An urgent change may bypass prior approval only under an explicitly configured policy and permission; the system shall record the reason and require a later review.	Medium	Planned
FR-6.5	A user shall not be able to approve their own change request.	Medium	Planned
FR-6.6	An approved change request shall be applied through the same validation as a direct update, and shall generate the corresponding event.	High	Planned

3.2.7 F7 — Incident and Audit Timeline

ID	Requirement	Priority	Status
FR-7.1	For each incident, the system shall show a chronological timeline of the report, related events, notifications, acknowledgements, decisions, approvals, task updates and resolution.	High	Planned
FR-7.2	Important changes shall record the acting user, action, entity, timestamp and before/after values where appropriate.	High	Planned
FR-7.3	Audit records shall be append-only for ordinary users and readable only by authorized roles.	High	Planned
FR-7.4	The timeline shall distinguish notification acknowledgement, task completion and incident resolution.	High	Planned
FR-7.5	The timeline shall be built from persisted records, not hard-coded demonstration events.	High	Planned

3.2.8 F8 — Overdue-Task Escalation

ID	Requirement	Priority	Status
FR-8.1	Action items may have a due time and a priority-based escalation rule.	High	Planned
FR-8.2	When a task passes its due time without resolution, the system shall mark it *Overdue* and notify the responsible department supervisor.	High	Planned
FR-8.3	Further escalation to Central Administration shall occur only when the configured rule requires it (e.g. a High-priority task still unresolved 30 minutes after escalation).	Medium	Planned
FR-8.4	The system shall record escalation and resolution times, and shall stop further escalation once the task is resolved.	Medium	Planned
FR-8.5	A scheduled server-side check shall evaluate overdue tasks at a configurable interval (default one minute).	Medium	Planned

Figure 3.1 shows the action-item and change-request life cycles as UML state machine diagrams.

3.2.9 F9 — Dashboards, Reports and Settings

ID	Requirement	Priority	Status
FR-9.1	Dashboard contents and navigation shall be determined by the authenticated role and department.	High	Planned
FR-9.2	Department dashboards shall show the department's authorized records, relevant notifications and assigned action items.	High	Planned

ID	Requirement	Priority	Status
FR-9.3	The Central Administrator dashboard shall show cross-department incidents, pending approvals, overdue tasks and acknowledgement status.	High	Planned
FR-9.4	The passenger *My Journey* page shall show only the signed-in passenger's journeys and notifications.	High	Planned
FR-9.5	Report values shall be derived from database records; any demonstration data shall be clearly labelled. Summary counts are currently database-derived; the sample-flight fallback is not yet labelled.	Medium	Partial
FR-9.6	Settings shall display the current user's role, department and application information without exposing secrets. Role display exists; department is planned.	Low	Partial

3.3 Key Features Summary

- One application, one login entry point, and a workspace per role and department.
- Clear ownership: departments edit only what they own and see only what they need.
- Public information without login; private passenger data visible only to its owner.
- Automatic, rule-based notifications and follow-up tasks across departments.
- Approval for sensitive changes, with a recorded decision.
- A complete, persisted incident and audit timeline.
- Overdue-task escalation to supervisors and, by rule, to central administration.

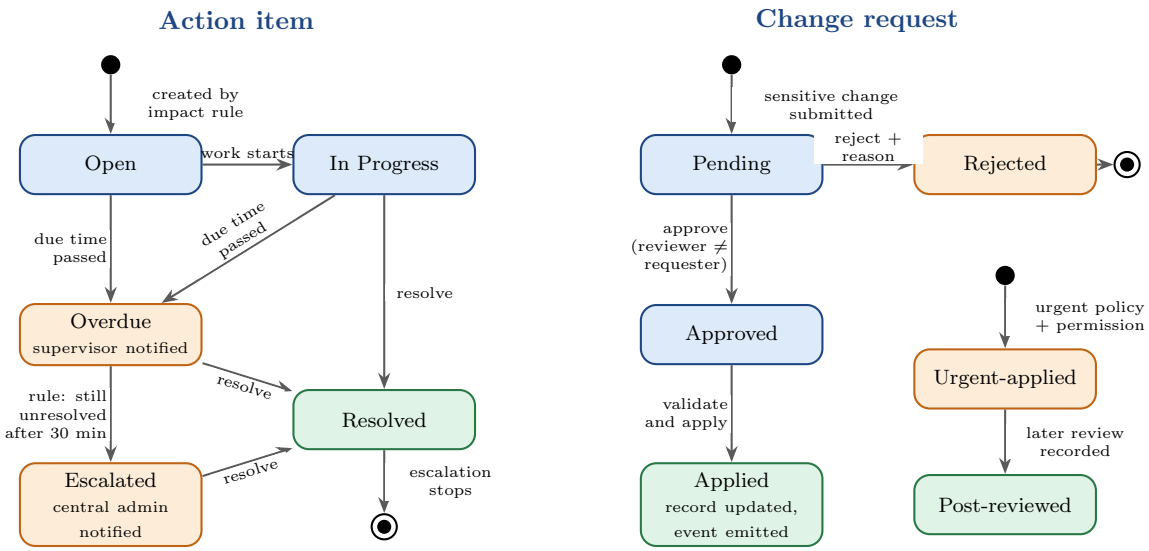


Figure 3.1: State diagrams — action item escalation (left) and change request approval (right)

4 Other Nonfunctional Requirements

Non-functional requirements constrain how the system delivers its functions — its performance, reliability, security and other quality attributes — rather than what it does.

4.1 Performance Requirements

ID	Requirement	Priority	Status
NFR-1	Under normal prototype load (up to 50 concurrent users), 95% of API requests shall complete within one second; compliance shall be verified by measurement.	Medium	To verify
NFR-2	The initial frontend page shall load within three seconds in a documented test environment.	Medium	To verify
NFR-3	A qualifying event shall produce its notifications and action items within five seconds under prototype load.	Medium	Planned

4.2 Safety and Reliability Requirements

ID	Requirement	Priority	Status
NFR-4	The backend shall validate request bodies and return consistent success/error responses.	High	Implemented
NFR-5	Important concurrent updates shall not silently overwrite newer data; version checks or conditional updates shall detect conflicts.	Medium	Planned
NFR-6	Event processing and notification creation shall be idempotent on retry and shall record failures that need recovery.	High	Planned
NFR-7	Deleting a record shall require explicit user confirmation and appropriate permission.	Medium	Implemented

4.3 Security and Privacy Requirements

ID	Requirement	Priority	Status
NFR-8	The system shall not expose passwords, JWT secrets, database credentials, internal incident payloads or another passenger's private data. The repository currently contains a committed `.env` file, which must be removed and its secrets rotated.	High	Partial

ID	Requirement	Priority	Status
NFR-9	Every protected operation shall enforce backend authorization, including direct API requests that bypass the frontend.	High	Implemented
NFR-10	Passwords shall be hashed with bcrypt; JWTs shall be signed with an environment secret and expire within 24 hours.	High	Implemented
NFR-11	Production deployment shall use HTTPS and restrict cross-origin requests to the frontend origin.	Medium	Planned

Security and privacy rules

- Public registration must never grant a privileged role.
- Department and resource authorization is checked on the server for every relevant request.
- Passenger records and notifications are scoped to the authenticated passenger.
- Public endpoints return allowlisted public fields only.
- Sensitive changes follow the approval workflow.
- Audit data is access-controlled and cannot be edited by ordinary users.
- Events and notifications carry references and minimal display fields, not unnecessary personal data.

4.4 Software Quality Attributes

ID	Requirement	Priority	Status
NFR-12	**Usability:** the interface shall be responsive and usable on modern desktop and mobile browsers, with consistent forms, tables and readable error messages.	Medium	Implemented
NFR-13	**Maintainability:** the code shall keep a modular separation of routes, middleware, controllers/services and models, and of frontend pages and components.	Medium	Implemented
NFR-14	**Transparency:** the application shall clearly distinguish live database records from seeded demonstration data.	High	Planned
NFR-15	**Extensibility:** a new department shall be addable through configuration (department, permissions, impact rules) and new pages, without changing the core authorization or event logic.	Medium	Planned
NFR-16	**Testability:** every requirement shall be traceable to at least one test case (Section 5.3).	Medium	Planned

4.5 Business Rules

- Every luggage, baggage-issue and transport record must reference an existing flight.

- A passenger can view only their own private journey and baggage details.
- A department may read relevant shared information without gaining write access to another department's records.
- A baggage report never changes the flight schedule automatically; Flight Operations decides.
- Notification acknowledgement does not equal task completion or incident resolution.
- A sensitive change remains pending until approved; the urgent path is available only when explicitly authorized and is always reviewed afterwards.
- A task is escalated only when its due time and rule are met; escalation stops when the task is resolved.
- A passenger alert must not state a missed connection or a confirmed alternative unless the data supports it.

5 Other Requirements

5.1 Database Requirements

The existing collections — **Users, Flights, Luggage, Visitors, Staff, Transportation** — are retained. Users gain `department` and `active` fields; Luggage gains an optional `passenger` reference. The following collections are proposed additions; they are not claims about the current database. Figure 5.1 shows the main entities and their relationships as a UML class diagram.

Table 5.1: Proposed new collections

Collection	Purpose and indicative fields
Departments	code, name, phase (core/phased), active
PassengerJourneys	passenger (User), segments (Flight references, order), status, timestamps
BaggageIssues	flight, optional luggage/passenger references, description, severity, status, reportedBy, incident
OperationalEvents	eventType, entityType, entityId, actor, department, payload (minimal), correlationId, createdAt
ImpactRules	eventType, conditions, recipient departments, actionRequired, actionDueMinutes, priority, active
Notifications	event, recipient user or department, message, readAt, acknowledgedAt, unique key (event + recipient)
ActionItems	event, incident, department, assignee, priority, status, dueAt, escalationLevel, escalatedAt, resolvedAt, resolution
ChangeRequests	entityType, entityId, proposed changes, requester, status (Pending/Approved/Rejected/Urgent-applied), reviewer, reason, decidedAt
AuditLogs	actor, action, entityType, entityId, before/after (where appropriate), timestamp, requestId
Announcements*	title, body, status (Draft/Published), author, publishedAt

* Phased with Passenger Services.

Data design rules

- Use references for shared records such as flights rather than independent copies of flight details.
- Minimize personal data in events and notifications; use references and allowlisted display fields.
- Use a unique key per event and recipient so that reprocessing cannot create duplicate notifications.
- Use record versioning or conditional updates for important concurrently edited records.

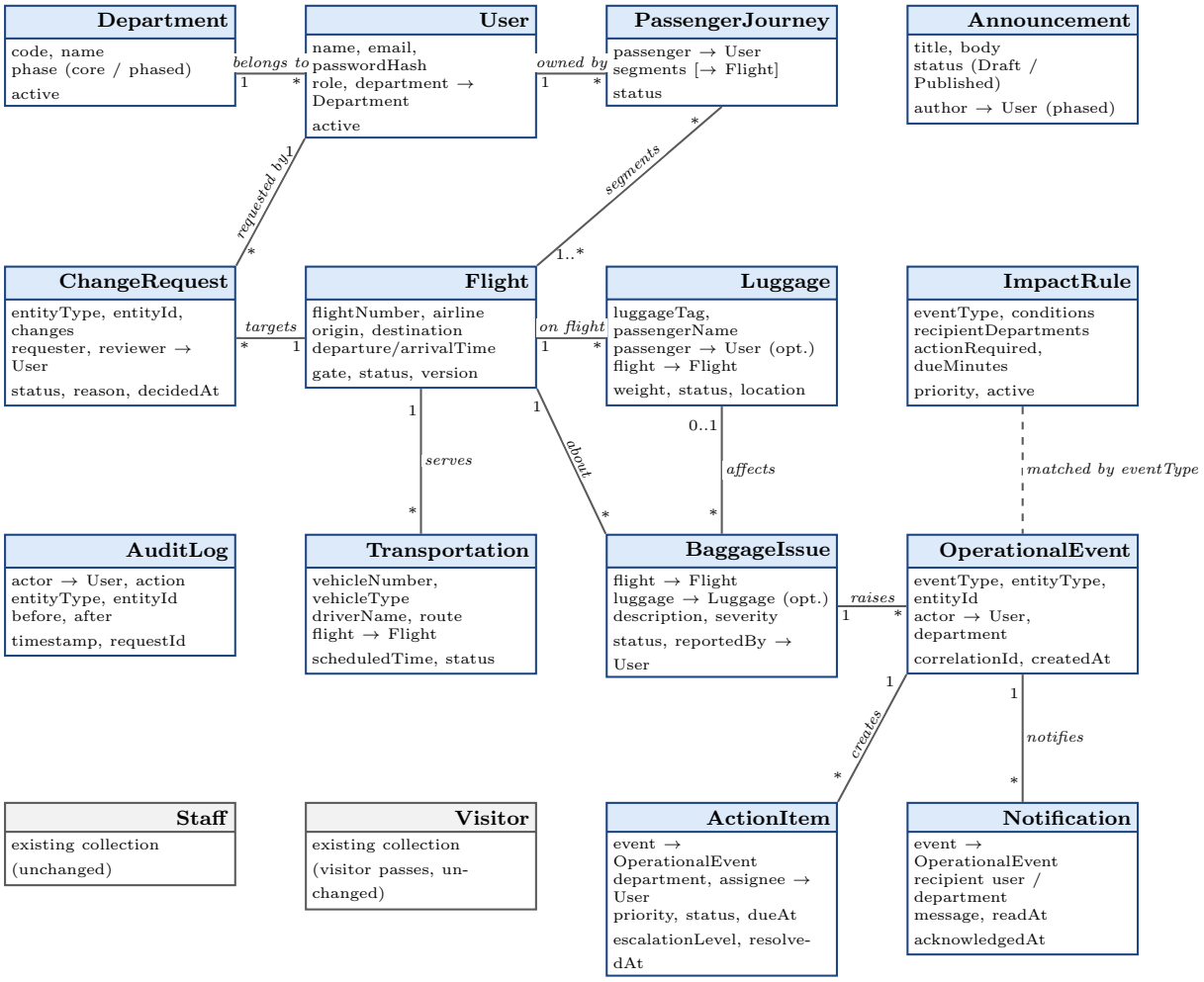


Figure 5.1: Class diagram of the main data entities

- Use database transactions for an operational update and its event where supported; otherwise implement and test a recovery strategy.

5.2 External Interface Requirements

User Interfaces

- **Public portal** — flight board, transport information, published announcements.
- **Passenger** — registration/login, My Journeys, My Notifications.
- **Department workspace** — department records, notifications, action items, and (for supervisors) team tasks and escalations.
- **Central administration** — overview, incidents and timelines, approvals, overdue tasks, users and roles.

Software Interfaces (REST API)

The endpoints below are proposed contracts. They shall be reconciled with the existing routes; working prototype endpoints (`/api/auth`, `/api/flights`, `/api/luggage`, `/api/visitors`, `/api/staff`, `/api/transportation`) are kept.

API group	Example endpoints	Access
Authentication	POST `/api/auth/register`, POST `/api/auth/login`, GET `/api/auth/me`	Public (register creates Passenger only); authenticated profile
Public information	GET `/api/public/flights`, `/api/public/transport`, `/api/public/announcements`	Public; allowlisted fields only
Passenger	GET/POST `/api/passenger/journeys`, GET `/api/passenger/notifications`	Passenger; own records only
Operations	`/api/flights`, `/api/luggage`, `/api/transportation`, `/api/baggage-issues`	Scoped by role, department, resource and field
Coordination	GET `/api/events`, GET `/api/notifications`, POST `/api/notifications/:id/acknowledge`, GET/PATCH `/api/action-items`	Recipient, owning department or supervisor
Approvals	POST `/api/change-requests`, POST `/api/change-requests/:id/approve`, POST `/api/change-requests/:id/reject`	Authorized requester / reviewer
Incidents	GET `/api/incidents/:id/timeline`	Involved departments; Central Administration
Administration	GET `/api/admin/overview`, GET/PATCH `/api/admin/users`, GET `/api/audit-logs`	Central / Super Administrator

Communication Interfaces

HTTP/HTTPS with JSON bodies between browser and API; the API connects to MongoDB using a connection string from environment configuration. Notifications are delivered in-app by the frontend polling the notifications endpoint (default every 30 seconds); WebSockets are a future enhancement.

5.3 Verification and Traceability

Test cases are written against the requirement IDs, giving the requirement-to-test traceability recommended by ISO/IEC/IEEE 29148. Tests for *Planned* requirements will be executed once those requirements are implemented and are not claimed as passing.

Test	Scenario	Expected result	Requirement
TC-01	Public user tries to create or update a flight	HTTP 401; no record change	FR-2.1, FR-2.6
TC-02	Public registration submits the `admin` role	Account created as Passenger, or rejected	FR-1.2
TC-03	Staff login	Redirected to own department workspace	FR-1.5, FR-9.1
TC-04	Baggage staff change a flight's gate	HTTP 403; flight unchanged	FR-2.3, FR-3.1
TC-05	Passenger requests another passenger's baggage	No private record returned	FR-2.4, FR-4.5
TC-06	Baggage issue reported on a flight	Issue and linked event persisted	FR-3.6, FR-5.1
TC-07	Same event processed twice	No duplicate notifications or action items	FR-5.3, NFR-6
TC-08	Notification acknowledged	Notification acknowledged; task still open	FR-5.4, FR-7.4
TC-09	Flight cancellation submitted	Flight unchanged while request is pending	FR-6.2
TC-10	Requester approves own change request	Rejected (HTTP 403)	FR-6.5
TC-11	Incident timeline opened	Recorded actions appear in order with actor and time	FR-7.1, FR-7.5
TC-12	High-priority task passes its due time	Marked Overdue; supervisor notified	FR-8.2
TC-13	Escalated task resolved	No further escalation recorded	FR-8.4
TC-14	Unpublished announcement requested publicly	Not returned	FR-4.1, FR-4.5
TC-15	Stale concurrent update submitted	Conflict detected; newer data kept	NFR-5
TC-16	Passenger with a journey on a delayed flight	Only that passenger sees a "potential impact" alert	FR-4.3, FR-4.4

5.4 Delivery Phases

1. Audit the existing source code and confirm what already works.
2. Define roles, departments and the server-side permission matrix; restrict public registration.
3. Add passenger journeys and targeted in-app notifications.
4. Complete the Flight Operations, Baggage Handling and Ground Transport workspaces and ownership rules.
5. Add persisted events, impact rules, notifications and action items.
6. Add the incident timeline and audit history.
7. Add the approval workflow and overdue-task escalation.
8. Run the end-to-end scenarios and update documentation; then add phased departments as time permits.

5.5 Acceptance Criteria for the Initial Expanded MVP

1. Existing core functions are audited and working functions are preserved.
2. Role and department permissions are enforced by the backend.
3. A passenger can register a test journey and receives only relevant in-app alerts.
4. A baggage issue linked to a flight produces an event and targeted department notifications.
5. Notification acknowledgement is tracked separately from task status.
6. The incident timeline shows report, notifications, decisions and resolution from persisted records.
7. An overdue task triggers a supervisor escalation according to a configured rule.
8. A sensitive change remains pending until an authorized decision is recorded.
9. Tests show that private passenger data and unauthorized operational fields cannot be accessed or modified.

5.6 Future Enhancements

- External airline/airport flight feeds and baggage-scanning integrations.
- Email/SMS notifications through a delivery provider.
- Expanded Maintenance and Passenger Services workflows.
- Real-time dashboard updates using WebSockets.
- Advanced analytics and exportable reports.
- Production hardening: monitoring, rate limiting, backup and recovery, and load testing.